

RetinaGuard — Technical Report

6.1 READINESS & CURRENT-STATE REPORT

Generated: September 24, 2026 · Target: Prompt 6.1 Baseline Verification · Target Branch: main

PROJECT: RetinaGuard (SIH 2026 / PS 26038)	BACKEND: Node.js 24 + Express + SQLite
SYSTEM: Edge Node Retinal Screening & AI Triage System	FRONTEND: Vite 8 + React 19 + TypeScript
READINESS VERDICT: TECHNICALLY READY TO BEGIN PROMPT 6.1 TEST STATUS: 14/15 suites passed (81/83 tests)	

A. Current Architecture

- Backend Architecture:** Node.js (v24.16.0), Express.js REST API with Socket.IO (/ws) event bus. Structured with dependency-injected service container (container.js) layered across routes -> controllers -> services -> repositories -> SQLite/Storage.
- Frontend Architecture:** Vite v8.2.2 SPA, React v19.2.8, React Router v7.18.2, TypeScript (~6.0.2), Tailwind CSS v4.3.3. Uses tokenized design system in index.css with bundled Oswald and Merriweather fonts.
- Database & Schema:** Embedded SQLite (better-sqlite3 v11.5.0, WAL mode) in data/retinaguard.db. 4 migrations applied (001_init, 002_triggers, 003_views, 004_diabetes_history). Tables verified: schema_migrations, users, refresh_tokens, patients, consultations, images, analysis_results, explainability, reviews, reports, sync_queue, audit_logs, idempotency_keys.
- API Routes & Controllers:** reports.routes.js, reports.controller.js, reportService.js, pdfService.js, reviews.routes.js, reviewerService.js, clinicalSafetyService.js, auditService.js, storageService.js, qrService.js.
- Authentication & RBAC:** JWT access tokens (15m TTL), refresh tokens (7d TTL, hashed SHA-256 in DB). RBAC enforced via authenticate and authorize(...roles). Admin silent bypass has been removed.

B. Relevant Functionality

The table below summarizes the verified implementation status for all core functional areas relevant to Prompt 6.1:

Area	Status	Evidence / File
PDF/Report Generation	IMPLEMENTED	pdfService.js, reportService.js
Clinical & Patient Variants	IMPLEMENTED	pdfService.js (dual variant rendering: 2-page clinical / 1-page patient)
Adjudication Gating	IMPLEMENTED	reportService.js assertAdjudicated() enforces review_complete or closed status
Report Download Flow	IMPLEMENTED	reports.controller.js (?download=1), ReportPage.tsx dual download buttons
Authentication & RBAC	IMPLEMENTED	auth.routes.js, authorize.js (roles: technician, reviewer, admin, district)
Audit Logging	IMPLEMENTED	REPORT_GENERATED recorded with SHA-256 digest in audit_logs table
SHA-256 Hashing	IMPLEMENTED	PDF buffer hashed via crypto sha256; auditRepository.verifyChain()
QR Generation	IMPLEMENTED	qrcode package generates QR with Case ID (no PHI) in PDF audit block
Banned-Word Protection	IMPLEMENTED	clinicalSafetyService.js (PROHIBITED_TERMS) enforced via assertSafeLanguage
Referral Urgency Handling	IMPLEMENTED	REFERRAL_URGENCY_MAP with bilingual English/Hindi actions in pdfService.js
Schema Field Mapping	IMPLEMENTED	reportService.js buildModel() implements SCHEMA_VERSION = "1.0.0"
Frontend Report Page	IMPLEMENTED	ReportPage.tsx with Print, Download Clinical, and Download Patient buttons
Seeded PDF Test Cases	IMPLEMENTED	002_pdf_test_cases.js seeds TEST-PDF-CASE-001, 002, 003, and 409 in DB

C. Previous Implementation Status

- PDF/Report Generation (pdfService.js & reportService.js): Resolved earlier issue where payload was undefined. PDFKit A4 rendering now maps the complete clinical model including patient demographics, screening metadata, AI triage results, and reviewer adjudication. Dynamic row heights in key-value grids prevent multi-line overlaps.
- Dual PDF Variants: Clinical variant produces 2 pages with full screening technical parameters, AI triage analysis card, Grad-CAM heatmap thumbnail, authoritative reviewer adjudication, referral urgency, and Section 6 audit block with tamper verification and QR code. Patient variant produces a 1-page clean summary containing demographics, screening outcome, bilingual (English + Hindi) referral actions, and patient advice bullets.
- Adjudication Gating: Gated in reportService.assertAdjudicated(). Strictly blocks generation and streaming unless consultation is in review_complete or closed status, throwing ClinicalSafetyError (HTTP 409).

4. SHA-256 Hashing & Audit Trail: Rendered PDF buffer is hashed via `crypto.createHash("sha256")` and recorded in `audit_logs` under `REPORT_GENERATED`. Tamper verification is verified through `auditRepository.verifyChain()`.

5. Seeded Test Cases: `002_pdf_test_cases.js` seeds TEST-PDF-CASE-001 (agreed), TEST-PDF-CASE-002 (modified), TEST-PDF-CASE-003 (abstained), and TEST-PDF-CASE-409 (awaiting review). Verified present in database.

D. Relevant Documentation Found

- **UPGRADE_REPORT.md**: 333-line log documenting UI tokenization, PDF blank-page bug resolution, removal of fake mock models, RBAC bypass fix, and validation hardening.
- **MIGRATION_NOTES.md**: Covers bundled offline fonts (@fontsource), frozen MATLAB interface contracts (contracts.js), database migrations, and geographic reference data.
- **retinaguard-backend/docs/API.md**: Narrative API specification including reports, verification endpoint `/reports/verify/:token`, and review workflow.
- **retinaguard-backend/docs/interfaces.ts**: TypeScript interfaces for ScreeningReport, Patient, Consultation, AnalysisResult, and Review.
- **SIH_26038_DR_Screening_Blueprint.md**: System clinical blueprint detailing ICDR classification, triage priorities (P0-P3), and non-negotiable screening vs diagnosis boundaries.

E. Tests / Typecheck / Build Evidence

- **Frontend Build**: PASSED clean (`tsc -b && vite build` executed in 3.82s with 0 errors). All 35 client assets generated.
- **Backend Test Suite**: 14 of 15 test suites passed; 81 of 83 tests passed. Integration test `tests/integration/e2ePipeline.test.js` passed completely (includes case lifecycle, human review, report generation, report JSON fetch, report PDF fetch, and audit trail).
- **Test Failure Note**: The 1 failing suite is `tests/unit/matlabService.test.js` (2 failed tests). This is a pre-existing, intentional mismatch caused by removing fake inference from `mockAdapter.js` (which now throws "DR grading model is not integrated" to protect clinical safety).

F. Current Git Changes

git status output shows working tree changes cleanly scoped to PDF reporting and landing cleanup:

- **Modified Files (10)**: `RetinaGuard-Frontend-v3/src/lib/http.ts`, `Landing.tsx`, `Login.tsx`, `ReportPage.tsx`, `api.ts`; `retinaguard-backend/package.json`, `package-lock.json`, `reports.controller.js`, `pdfService.js`, `reportService.js`.
- **Untracked Files (2)**: `retinaguard-backend/src/assets/` (NotoSansDevanagari fonts) and `retinaguard-backend/src/database/seeds/002_pdf_test_cases.js`.
- **Assessment**: All 10 modified files directly relate to the PDF reporting enhancements, dual variants, and landing polish. No unrelated changes.

G. Blockers & Open Issues

- **Known Test Failure**: `matlabService.test.js` expects obsolete fake model responses; needs update to assert abstention path.
- **Font Assets**: NotoSansDevanagari font files sit under untracked directory `src/assets/fonts/`. Must remain intact for Hindi rendering.
- **Port Alignment**: Frontend `http.ts` has been aligned to backend default port 4000.
- **Dedicated PDF Test Suite**: No standalone unit test suite for `pdfService.js` (`e2ePipeline.test.js` tests it at integration level).

H. Files Likely Relevant When Implementing Prompt 6.1

- **pdfService.js**: Core PDF document layout, fonts, bilingual rendering, QR, section blocks.
- **reportService.js**: Report model assembly, adjudication gating, SHA-256 digest creation, audit recording.
- **reports.controller.js & reports.routes.js**: Variant query parameters (`?variant=patient|clinical`), inline vs download headers.
- **clinicalSafetyService.js**: Prohibited clinical terms and triage priority logic.
- **002_pdf_test_cases.js**: Seeded test scenarios (TEST-PDF-CASE-001, 002, 003, 409).
- **ReportPage.tsx & api.ts**: Frontend report interface with Print and Dual PDF download triggers.

I. Overall Readiness Verdict

VERDICT: THE REPOSITORY IS TECHNICALLY READY TO BEGIN PROMPT 6.1

- Frontend compiles cleanly (`tsc -b && vite build` passed with 0 errors).
- Edge SQLite database is fully migrated and seeded with test cases.
- Core PDF rendering architecture, dual variants, adjudication gating, SHA-256 hashing, and QR generation are verified.